Bayesian Optimization as a Budget-Aware Local Optimizer within a Continuous EDA

Aimar Barrena Pol

Optimization Techniques

Abstract. This work evaluates Bayesian optimization (BO) as a local search (LS) operator within a continuous estimation of distribution algorithm (EDA). A univariate Gaussian EDA is compared with two hybrids using LS with L-BFGS-B and Gaussian-process BO with expected improvement (EI). All methods are evaluated on Sphere, Rastrigin, Ackley, and Rosenbrock in five dimensions, using ten seeds and exactly 600 objective evaluations per run. Local refinement is restricted to the best population member and receives at most ten evaluations per generation. The plain EDA achieves the lowest median final objective on all four benchmarks. BO improves on the median performance of the L-BFGS-B hybrid on three benchmarks, but does not compensate consistently for the population evaluations it replaces. These results show that adding surrogate-guided refinement is not automatically beneficial under a small shared evaluation budget. The conclusions concern this specific short-budget implementation; they do not establish a general ranking of local optimizers.

Keywords: Estimation of Distribution Algorithms · Bayesian Optimization · Local search · Evaluation budget

1 Problem Definition

This work investigates whether Bayesian optimization (BO) can improve the performance of a continuous estimation of distribution algorithm (EDA) when used as a local search (LS) operator. The problem is to find high-quality solutions within a fixed number of objective evaluations.

1.1 Optimization Setting

We consider bounded continuous minimization problems of the form

$$\min_{\mathbf{x}\in\mathcal{X}} f(\mathbf{x}), \qquad \mathcal{X} = \prod_{j=1}^{d} [\ell_j, u_j], \tag{1}$$

where $\mathbf{x}$ is a vector of d real-valued decision variables, f is the objective function, and ℓ_j and u_j define the bounds of each variable. The algorithms obtain information about the objective through evaluations of candidate solutions. Given a

budget B, the desired output is the best solution observed before that budget is exhausted:

$$\hat{\mathbf{x}}_B \in \underset{\mathbf{x}\in\{\mathbf{x}_1,\ldots,\mathbf{x}_N\}}{\arg\min} f(\mathbf{x}), \qquad N \leq B. \tag{2}$$

The budget includes evaluations used by both population search and LS, including numerical gradient estimation when applicable.

The experimental scope is restricted to four deterministic benchmark functions in five dimensions: Sphere, Rastrigin, Ackley, and Rosenbrock. These provide different search landscapes on which to examine the proposed algorithms.

1.2 Motivation for Local Refinement

EDAs learn a probability distribution from selected solutions and sample new candidates from it [2]. The baseline uses independent Gaussian marginals, which concentrate sampling around promising regions but do not represent dependencies between variables. Local refinement may improve precision, but it also consumes additional evaluations that could be used for population search. BO adds a surrogate that can guide these local evaluations, while reducing the need for expensive function evaluations compared to traditional methods. The experiment tests whether that tradeoff is favorable.

1.3 Research Objective and Comparison

The central research question is whether BO-based local refinement produces better solutions within the same objective-evaluation budget than either a plain Gaussian EDA or an EDA using L-BFGS-B LS. The plain EDA provides the population-search baseline, while L-BFGS-B provides a conventional local-optimization comparison. Together, these methods allow the experiments to assess both the value of adding LS and the relative performance of the two LS choices.

Performance is examined through the final best objective value and variability across repeated runs. Evaluation histories are additionally used to compare the convergence of the algorithms. Equal evaluation budgets are the principal comparison constraint. Conclusions are limited to the selected benchmarks, dimensionality, and experimental settings, rather than establishing that one approach is universally preferable.

2 Steps for Solving the Problem

To solve the problem, the experiment implements three algorithms: a plain Gaussian EDA, a hybrid EDA with classical L-BFGS-B LS, and a hybrid EDA with BO-based LS. The three algorithms share the same population search, selection ratio, and population size. The hybrids differ only in their LS operators. Each run uses a fixed budget of 600 objective evaluations, which are shared between the population and local components. The hybrids allocate 30 evaluations

to each EDA iteration and 10 evaluations to the local component, while the plain EDA uses all evaluations for its population search. All three methods share the same limit of 600 evaluations.

2.1 Gaussian EDA as the Population Search

The implementation uses the `GaussianUMDA` interface from `pateda`. Its search model is a product of univariate normal distributions. If S_t denotes the selected individuals in generation t, the basic model can be expressed as

$$p_t(\mathbf{x}) = \prod_{j=1}^{d} \mathcal{N}(x_j; \mu_{t,j}, \sigma_{t,j}^2), \quad \mu_{t,j} = \frac{1}{|S_t|} \sum_{\mathbf{x} \in S_t} x_j, \tag{3}$$

where each variance describes the spread of the selected values in one coordinate. This factorization is computationally simple, but it cannot represent correlations between coordinates. That limitation is relevant on landscapes such as Rosenbrock, where good solutions follow a curved valley.

Each run uses a population of 30, selects the best half, and enables elitism. The initial population and subsequent generations are managed by the EDA framework within the specified bounds. Because the framework maximizes fitness, the minimization objective is supplied as $F(\mathbf{x}) = -f(\mathbf{x})$. All reported results are converted back to the original objective, so smaller values always indicate better solutions. The plain method evaluates its initial population and 19 subsequent populations, consuming $30 + 19 \times 30 = 600$ evaluations.

2.2 Shared LS Interface

The two hybrids retain the same population model, selection ratio, population size, and elitism setting. An LS component is added to each hybrid. That component receives a population and its evaluated fitness values, selects the member with the highest fitness, and attempts to improve it. The selected individual is replaced only if a strictly better evaluated candidate is found. Its original fitness is retained otherwise. This rule prevents a local call from directly worsening that individual, although the hybrid can still finish worse than the plain EDA because it allocates evaluations differently.

Both hybrids evaluate 14 generations after initialization, due to the limit of 600 evaluations and the allocation of 30 evaluations to each EDA iteration and 10 evaluations to the local component.

$$G_{\text{hybrid}} = \left\lceil \frac{600}{30 + 10} \right\rceil - 1 = 14. \tag{4}$$

The LS component is attached through the framework's internal component interface. In each iteration it receives the remaining global budget and is capped at ten evaluations, both per generation and per selected solution. With only one selected solution, these two local limits coincide.

2.3 Classical LS with L-BFGS-B

The classical operator calls `scipy.optimize.minimize` with the L-BFGS-B method and the original global bounds. L-BFGS-B is a bound-constrained quasi-Newton method. No analytic Jacobian is supplied, so gradients are estimated numerically. The implementation sets both `maxiter` and `maxfun` to the available local budget. These parameters alone are insufficient to enforce a strict objective-call limit when numerical gradients are used, as documented by SciPy [4].

An additional guard therefore returns a large penalty, 10^{100}, after ten true local evaluations have been consumed. These subsequent penalty returns do not evaluate the benchmark. The wrapper tracks the best actual point visited and returns that point, rather than relying on the optimizer's final status or final iterate. This protects the evaluation budget but can interrupt a gradient estimate or line search. Accordingly, the experiment evaluates a tightly budgeted L-BFGS-B wrapper, rather than L-BFGS-B run to convergence.

2.4 BO with EI

BO constructs a probabilistic surrogate from evaluated points and uses an acquisition function to choose the next evaluation [1]. Here each local call creates a new `skopt.Optimizer` with a Gaussian-process surrogate and expected improvement (EI). Gaussian processes provide predictive means and variances from a covariance model [3]. The notebook leaves the detailed covariance and fitting settings at the library defaults; these are not independently tuned in this study.

The search region is a fixed box around the selected center $\mathbf{c}$:

$$a_j = \max\{\ell_j, c_j - 0.05(u_j - \ell_j)\}, \qquad b_j = \min\{u_j, c_j + 0.05(u_j - \ell_j)\}. \tag{5}$$

Thus, the radius along each coordinate is 5% of the global range. Away from a global boundary, the full local width is 10% of the global range. This is a fixed local box, not an adaptive trust-region algorithm.

The already evaluated center and its objective value initialize the optimizer through `tell`; no extra benchmark call is needed. Setting `n_initial_points=1` makes this observation satisfy the initial-design requirement. Each of the next ten iterations asks for a candidate, evaluates its true objective, and updates the surrogate. For minimization, EI can be written as

$$\mathrm{EI}(\mathbf{x}) = \mathbb{E}[\max\{0, f_{\min} - f(\mathbf{x}) - \xi\} \mid \mathcal{D}], \tag{6}$$

where $\mathcal{D}$ contains the observations, $f_{\min}$ is the best observed local value, and ξ is the acquisition's exploration parameter. The implementation uses the library's default acquisition parameters. A new surrogate is created at every local invocation, so previous generations' observations are not reused. Each completed call therefore has only the center and at most ten newly evaluated observations available to guide refinement.

3 Experimental Design

3.1 Benchmark Suite

The experiments use four deterministic continuous functions in $d = 5$ dimensions. The conventional definitions are given below, with the scalar domain bounds applied to every coordinate:

$$f_{\text{Sphere}}(\mathbf{x}) = \sum_{j=1}^{d} x_j^2, \tag{7}$$

$$f_{\text{Rastrigin}}(\mathbf{x}) = 10d + \sum_{j=1}^{d} \left[x_j^2 - 10\cos(2\pi x_j) \right], \tag{8}$$

$$f_{\text{Ackley}}(\mathbf{x}) = -20\exp\left(-0.2\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2} \right) - \exp\left(\frac{1}{d}\sum_{j=1}^{d} \cos(2\pi x_j) \right) + 20 + e, \tag{9}$$

$$f_{\text{Rosenbrock}}(\mathbf{x}) = \sum_{j=1}^{d-1} \left[100(x_{j+1} - x_j^2)^2 + (1 - x_j)^2 \right]. \tag{10}$$

The notebook imports the benchmark implementations from `pateda`. Sphere and Rastrigin use $[-5.12, 5.12]^5$, Ackley uses $[-32.768, 32.768]^5$, and Rosenbrock uses $[-5, 10]^5$ as the domain bounds. The global minimum is zero for each conventional function: at the origin for the first three, and at the all-ones vector for Rosenbrock. Final objective values can consequently be read as residual errors relative to these optima.

Sphere is a smooth separable baseline. Rastrigin introduces many local minima while retaining coordinate separability. Ackley combines oscillations with a central basin and a broad outer region. Rosenbrock introduces strong interactions between adjacent variables and a narrow curved valley. Together they test different difficulties.

3.2 Controlled Settings and Repetition

Table 1 summarizes the common settings and the differences introduced by the local operators. There are ten runs for each method–benchmark combination, using seeds 0 through 9. This gives 120 runs in total and 72,000 recorded objective evaluations.

Table 1. Experimental settings used in the notebook.

Setting	Value
Decision variables	5 continuous variables
Population and selection	30 individuals; best 50% selected
Elitism	Enabled for all three methods
Objective budget	600 true evaluations per run
Repetitions	10 seeds per benchmark and method
Local subset	Best individual in the population
Local allowance	At most 10 evaluations per invocation
Classical search	L-BFGS-B; numerical gradients; global bounds
BO search	Gaussian process; EI; fixed local box
BO box radius	5% of the global range per coordinate

Identical seed labels support descriptive comparisons but do not imply identical candidate populations after the methods diverge.

4 Results: Final Objective Values and Convergence

4.1 Final Objective Values

Figure 1 compares the final objective values across ten runs, and Table 2 summarizes the same distributions using medians and interquartile ranges. Lower values are better. The plain EDA obtains the lowest median on all four benchmarks. Its advantage is largest on Sphere, where the median is 3.55×10^{-5}, compared with 3.12×10^{-4} for BO and 5.02×10^{-4} for L-BFGS-B. On Ackley, its median is 0.0897, compared with 0.417 and 0.484 for the classical and BO hybrids, respectively.

The distributions overlap more on Rastrigin and Rosenbrock. On Rastrigin, the medians are 6.44 for the plain EDA, 7.44 for BO, and 8.11 for L-BFGS-B; on Rosenbrock, they are 4.61, 5.49, and 7.16, respectively. BO therefore achieves a lower median final objective value than the classical hybrid on three benchmarks, with Ackley as the exception, but neither hybrid improves on the plain EDA by this measure. The spread of the individual points shows that the ranking can differ between runs.

Table 2. Final objective after 600 evaluations. Each entry gives the median, with the interquartile range below. The lowest median per benchmark is bold.

Benchmark	Plain EDA	EDA + L-BFGS-B	EDA + BO
Sphere	$\mathbf{3.55\times10^{-5}}$	5.02×10^{-4}	3.12×10^{-4}
	$[9.68\times10^{-6}, 7.38\times10^{-5}]$	$[3.81\times10^{-4}, 6.17\times10^{-4}]$	$[2.36\times10^{-4}, 6.45\times10^{-4}]$
Rastrigin	**6.44**	8.11	7.44
	[4.81, 9.2]	[7.17, 11.4]	[5.64, 8.8]
Ackley	**0.0897**	0.417	0.484
	[0.0486, 0.0999]	[0.253, 0.804]	[0.388, 0.525]
Rosenbrock	**4.61**	7.16	5.49
	[3.44, 6.01]	[3.82, 9.74]	[3.56, 6.83]

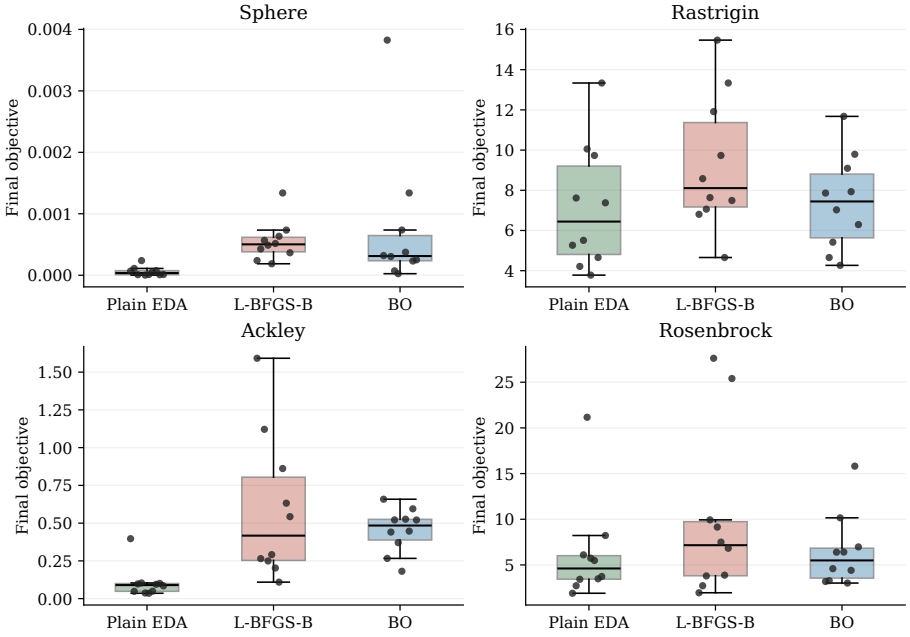


Fig. 1. Final objective values after 600 evaluations across ten seeds. Boxes show the interquartile range and median; points show all runs. Each benchmark uses its own linear vertical scale. Whiskers extend to the most extreme values within 1.5 interquartile ranges of the box.

4.2 Convergence

Figure 2 shows the evolution of the best objective found. The horizontal axis counts all objective evaluations, including those used by LS. This makes the three methods comparable despite their different numbers of EDA generations. Each curve is the mean across ten runs, and the shaded band is the interquartile range.

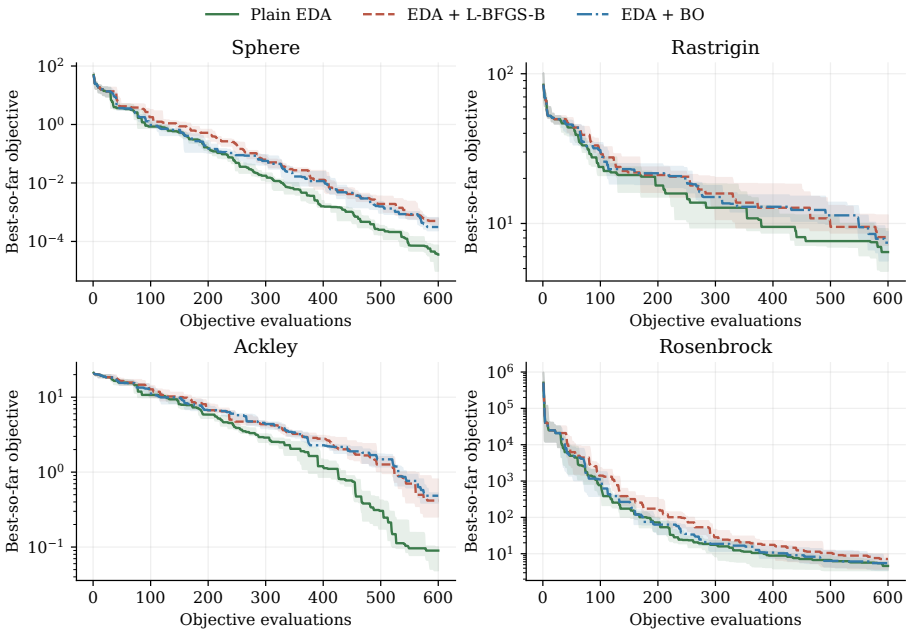


Fig. 2. Best-so-far objective over 600 evaluations. Lines show means and bands show interquartile ranges across ten seeds. Each panel uses its own logarithmic vertical scale.

All three methods reduce the objective during the run, but the LS variants do not produce a better mean endpoint. On Sphere, the plain EDA finishes about one order of magnitude below the hybrids. On Rastrigin, the mean evaluation at which each run first finds its final best value is 505 for the plain EDA, 536 for L-BFGS-B, and 538 for BO. This leaves a longer final period without improvement than on the other benchmarks. On Sphere, Ackley, and Rosenbrock, the corresponding mean evaluations range from 571.5 to 594, so improvements are still found near the end of the budget.

No run reaches an objective value of 10^{-6} or lower. The observed plateaus therefore indicate a lack of further improvement within the available budget, rather than convergence to the global optimum. Under these settings, using all evaluations for population search gives the best mean final performance. Local

refinement can improve the selected individual, but its evaluation cost does not translate into a better overall mean result in these experiments.

5 Conclusions

This work evaluates BO as an LS operator within a continuous Gaussian EDA. Under the same budget of 600 objective evaluations, the plain EDA obtains the lowest median final objective on all four benchmarks. BO gives lower median values than L-BFGS-B on Sphere, Rastrigin, and Rosenbrock, while L-BFGS-B performs better on Ackley. However, neither LS variant improves on the plain EDA by this measure. For the tested settings, assigning evaluations to local refinement is therefore less effective than using them for population search.

The convergence histories show that all three methods improve during the run, with improvements still occurring near the end of the budget on several benchmarks. No run reaches the objective tolerance of 10^{-6}, so the final plateaus do not establish convergence to the global optimum. These findings are limited to the four five-dimensional benchmarks, ten seeds, and the local allowance of ten evaluations. Further experiments could vary the total budget and the number of evaluations assigned to LS to examine when local refinement becomes beneficial.

References

1. Frazier, P.I.: A tutorial on Bayesian optimization. arXiv preprint arXiv:1807.02811 (2018). https://doi.org/10.48550/arXiv.1807.02811
2. Larrañaga, P., Lozano, J.A. (eds.): Estimation of Distribution Algorithms: A New Tool for Evolutionary Computation. Kluwer Academic Publishers (2002)
3. Rasmussen, C.E., Williams, C.K.I.: Gaussian Processes for Machine Learning. MIT Press (2006), https://gaussianprocess.org/gpml/
4. SciPy Developers: minimize with method L-BFGS-B. SciPy optimization documentation, https://docs.scipy.org/doc/scipy/reference/optimize.minimize-lbfgsb.html, accessed 29 October 2026